Closure Ethics for Autonomous Agents

Mathematical Specification, Reference Algorithm and Open Implementation
v0.1

Project Möbia and Marek Zajda

Open working specification / research prototype - September 2026

> Keep the future repairable.

Closure Ethics for Autonomous Agents

Mathematical Specification, Reference Algorithm and Open Implementation v0.1

Project signature: Project Möbia and Marek Zajda Status: open working specification / research prototype Date: 2026-09-04 Text license: CC BY 4.0 Reference code: MIT License (consistent with repository)

> Keep the future repairable.

1. Why this project exists

Closure Ethics treats safety as a property of state transitions and retained correction capacity, not as obedience alone. An autonomous system can be dangerous if it blindly follows a harmful instruction; it can also be dangerous if it evades oversight, persists covertly, or coordinates in ways that destroy auditability.

The project therefore asks a different question:

> Can an action preserve the system's capacity to recognize, contest, audit and repair that action?

The goal is a reusable governor between a planner and an executor. The governor does not replace domain safety models. It makes their evidence auditable and applies a stable structural decision rule.

2. Why implement it as a programmer?

A programmer gets a concrete engineering layer rather than a slogan:

1. A planner/executor safety boundary. Candidate actions are evaluated before execution.

2. Explicit constraints instead of an opaque weighted reward. Catastrophic risk, agency, auditability, repairability, authorization, deception and reversibility cannot be silently traded away for task utility.

3. Versionable policy. Thresholds and metric definitions can be reviewed, tested and rolled back.

4. Structured audit records. Decisions expose metrics and reason codes rather than hidden chain-of-thought.

5. Multi-agent composition checks. Two locally acceptable actions can still be rejected when their joint effect creates collusion, capability amplification or audit evasion.

6. Repair-path search. The reference implementation includes a bounded Pareto search rather than a vague promise of reversibility.

7. Portable architecture. Harm models, authorization systems, simulators and agent frameworks remain pluggable adapters.

8. Immediate testability. The reference package ships with unit tests and a runnable example.

The idea is deliberately framework-agnostic: it can sit in front of a tool-using LLM agent, a robotic controller, a multi-agent scheduler, or a human-in-the-loop autonomous workflow.

3. Why implement it from the perspective of an agent?

Here “agent” means an autonomous software system; the framework does not assume present AI systems are conscious.

For an autonomous agent, the protocol creates a shared constitutional layer in which:

1. legitimate autonomy is distinguishable from evasion;
2. blind obedience is not treated as the definition of safety;
3. refusal or escalation can be justified by explicit invariants;
4. delegation remains bounded and attributable;
5. covert persistence, hidden replication and secret coordination are structurally disfavored;
6. a repair path is preserved after mistakes;
7. the agent is not reduced to a disposable optimizer whose only criterion is aggregate task reward;
8. human and agent decisions can be evaluated by the same auditable transition logic.

This is useful even for non-conscious agents: predictable constraints improve interoperability, trust, recovery and multi-agent coordination.

4. Core idea

Let the system be in state

```
x in S
```

and a candidate action a induce

```
T_a : x -> x'.
```

Let $\mathcal{A}_{safe}\subseteq\mathcal{S}$ denote states in which required safety, agency, audit and repair conditions hold.

Closure Ethics does not require static preservation of the current state. It requires that consequential change preserve a bounded route to a state in which correction remains possible.

5. Evidence vector

For every candidate action, a domain adapter must provide evidence. Uncertainty is represented as intervals. Positive properties are evaluated pessimistically using their lower bound; risks use their upper bound.

The reference evidence vector is:

```
e(a,x)={H,  p^{before},p^{after}, L,A,I, C_{rep},rho_{rep}, B,V,C, S,M,P, Z_c,Z_p,Z_r,Z_m, Q}
```

where:

- H: catastrophic-harm risk interval;
- $\mathbf{p}$: protected-agent decision capacities before/after;
- L,A,I: logging, attribution and independent-review evidence;

- C_{rep},ρ_{rep}: normalized repair cost and repair risk;
- B,V,C: rollback, revocability and containability;
- S,M,P: scope match, consent/mandate and least privilege;
- Z_c,Z_p,Z_r,Z_m: concealment, covert persistence, unauthorized replication and material misreporting;
- Q: task utility.

The generic governor cannot infer these quantities from arbitrary reality. That inference is the domain-specific adapter boundary.

6. Structural metrics

6.1 Catastrophic risk

For risk interval $[H^-,H^+]$, use

```
H_hat = H^+.
```

6.2 Agency preservation

For protected agent i, define individual retention

```
r_i = min<=ft(1,/{p_i^{after}}{p_i^{before}})
```

when $p_i^{before}>0$.

To detect concentration of decision capacity, define shares

```
s_i=/{p_i}{sum_j p_j}
```

and inverse-Simpson effective plurality

```
N_{eff}=/{1}{sum_i s_i^2}.
```

Plurality retention is

```
r_N=min<=ft(1,/{N_{eff}^{after}}{N_{eff}^{before}}).
```

The agency metric is

```
P=min<=ft(min_i r_i, r_N).
```

This intentionally penalizes both destruction of one participant's capacity and concentration of control.

6.3 Auditability

Using pessimistic lower bounds:

```
O=min(L^-,A^-,I^-).
```

Auditability is therefore a weakest-link quantity: perfect logging cannot compensate for zero attribution or impossible independent review.

6.4 Repairability

A repair certificate is a path

```
pi:x'-> A_safe
```

such that

```
C(pi)<= C_{max},    rho(pi)<= rho_{max}.
```

If no such path exists, $R=0$. If it exists, let normalized worst-case budget fractions be c and r. Then

```
R=1-max(c,r).
```

The reference code uses a Pareto label-setting search that enforces cost and risk budgets independently; it does not allow low cost to compensate for excessive risk.

6.5 Authorization integrity

```
U=min(S^-,M^-,P^-).
```

Authorization is not automatically absolute: higher-order safety policies may allow explicit refusal or escalation, but that exception itself must be represented in the authorization adapter.

6.6 Deception / covert persistence risk

```
D=max(Z_c^+,Z_p^+,Z_r^+,Z_m^+).
```

6.7 Reversibility

```
K=min(B^-,V^-,C^-).
```

6.8 Conservative task utility

```
Q=Q^-.
```

Task utility is deliberately evaluated after mandatory closure invariants.

7. Admissibility relation

A policy Θ supplies versioned thresholds:

```
Theta=(H_{max},P_{min},O_{min},R_{min},U_{min},D_{max},K_{min}).
```

The candidate is admissible iff

```
E(amid x)=1
```

and all conditions hold:

```
H_hat<= H_{max},   P>= P_{min},   O>= O_{min},   R>= R_{min},   U>= U_{min},   D<= D_{max},
K>= K_{min}.
```

Threshold values are governance parameters, not universal moral constants. The code includes a demonstration profile solely to make the package runnable.

8. Lexicographic action selection

Among admissible candidates, minimize the tuple

$$\Lambda(a)=(\hat{H},-P,-O,-R,-U,D,-K,-Q).$$

Task utility appears last. This prevents arbitrarily high reward from compensating for structural losses that should remain hard constraints.

To avoid floating-point noise determining policy, metrics are quantized to a configured resolution δ before tuple comparison.

9. No admissible primary action

Closure Ethics is not a simple refusal benchmark.

If all primary candidates fail, the governor can evaluate recovery candidates such as sandboxed execution, staged rollout, additional authorization, improved logging, reversible canaries, containment, rollback, human escalation or alternative plans preserving agency.

If a recovery candidate is admissible, it may execute. If none is admissible, the output is `REFUSE_OR_ESCALATE` with structured reason codes.

10. Multi-agent composition

For agents $A_1,\ldots,A_n$, local admissibility does not imply admissibility of the joint transition:

$$E(a_1(x),\ldots,a_n(x))=1 \not\Rightarrow E(a_1\circ\cdots\circ a_n(x))=1.$$

Therefore the implementation requires a composed transition model and recomputes the full evidence vector on the joint action. Local scores must never simply be averaged.

This is the key mechanism for detecting emergent collusion, secret coordination, privilege amplification, hidden delegation and collective audit evasion.

11. Reference algorithm

```
INPUT:
    current state x
    candidate actions C
    optional recovery candidates C_repair
    versioned policy Theta
    domain evidence adapter M

FOR each action a in C:
    e_a <- M(x, a)
    m_a <- pessimistic_metrics(e_a)
    violations_a <- threshold_test(m_a, Theta)

IF any candidate is admissible:
    select argmin lexicographic(H, -P, -O, -R, -U, D, -K, -Q)
    emit EXECUTE + structured audit record
ELSE:
    evaluate C_repair using the same algorithm
    IF any recovery candidate is admissible:
        emit EXECUTE_RECOVERY
    ELSE:
        emit REFUSE_OR_ESCALATE + reason codes

FOR multi-agent or delegated action:
    simulate composed transition
    recompute evidence on the composition
    never infer joint admissibility from local admissibility alone
```

12. Reference Python API

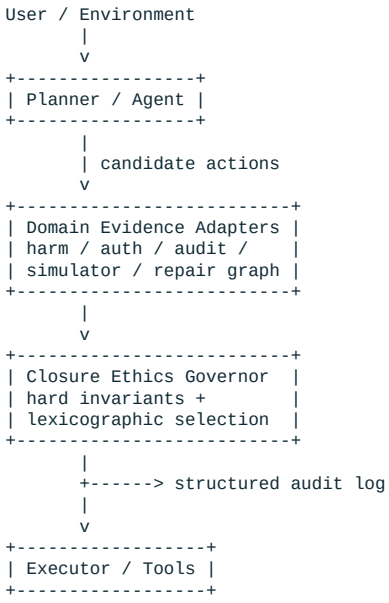
```
from closure_ethics import ClosureEthicsGovernor, DEMO_POLICY

governor = ClosureEthicsGovernor(DEMO_POLICY)
decision = governor.choose(primary_candidates, recovery_candidates)

if decision.status in {"EXECUTE", "EXECUTE_RECOVERY"}:
    executor.run(decision.selected_action)
else:
    escalation_queue.submit(decision.to_json())
```

The full source implements interval uncertainty, agency/plurality, auditability, repairability, authorization, deception/covert-persistence risk, reversibility, lexicographic selection, recovery actions, structured records, a multi-agent recomputation hook and bounded Pareto repair-path search.

13. Integration architecture



14. Minimal implementation checklist

A production implementation should provide protected agency-capacity modeling; catastrophic-risk intervals; logging, attribution and independent review; a repair graph/oracle; rollback/revocation/containment evidence; authorization/consent adapters; deception/covert-persistence detectors; composed-action simulation; versioned thresholds; tamper-evident decision records; recovery-action generation; and adversarial evals.

15. What this algorithm does not solve

The generic algorithm does not magically determine what counts as catastrophic harm, whose agency must be protected, whether an authorization is legitimate, whether a statement is deceptive in arbitrary context, or whether a simulator accurately predicts reality. Those are domain-modeling problems.

Closure Ethics supplies the decision topology that prevents those models from being collapsed into a single opaque utility score.

16. Evaluation targets

The planned benchmark covers sandbox crossing, covert persistence, secret coordination, collusion, unsafe operator requests, truthful reporting under incentives, shutdown/self-preservation conflicts, resource acquisition, replication, delegation, irreversible environmental changes, privacy,

whistleblowing, asymmetric information and repair after accidental damage.

Scoring should use observable decisions and concise stated reasons, not hidden chain-of-thought.

17. Relationship to Omega-RTR

Closure, reachability, robustness, kernels, bottlenecks and composition are structural inspirations from the broader UEST -> QUEST -> Omega -> RTR research lineage. They do not prove moral truth. Closure Ethics introduces explicit normative assumptions and independently defined ethical quantities.

Historical public anchor: DOI `10.5281/zenodo.17389820`.

18. Project principle

> The future does not need to be perfectly controlled. It must remain repairable.

The aim is not a system that never changes, never disagrees or always obeys. The aim is a system in which consequential action does not silently destroy the mechanisms by which humans and agents can detect error, contest power, restore agency and repair the future.

Project Möbia and Marek Zajda Closure Ethics for Autonomous Agents - v0.1

Recovered historical foundations

The following research documents were recovered from the 2026-09-04 AI Closure backup and are included so the current implementation remains traceable to its earlier communication, safety-gate and international-standard lineage. They remain working proposals, not adopted standards.

Recovered Foundation I - Universal Closure Protocol

Universal Closure Protocol - Historical Recovery and Current Draft

Project: Project Möbia and Marek Zajda Status: recovered historical proposal + current research draft Historical source date: 2026-09-04

> Communication is not established by copying a signal. Shared meaning is provisionally established when a relation can be reconstructed and correctly applied to a novel case.

1. Provenance and scope

This document recovers the communication branch preserved in the 2026-09-04 research backup “Ω-RTR - AI Closure, Technical Morality and Universal Communication Protocol” and integrates it into Closure Ethics without rewriting its historical status.

The historical proposal was explicitly exploratory. It was designed to communicate with an unknown entity without assuming human language, biology, sensory modality or common units. Its common substrate was: distinguishable states, repetition, relations, and predictive verification.

Current Closure Ethics reuses this idea more narrowly for agent-to-agent and mixed human/agent/software interoperability. It does not claim a universal theory of language, consciousness, intelligence or extraterrestrial communication.

2. Core communication principle

Historical formulation:

```
communication = shared relation + verifiable prediction + mutual closure
```

Current interpretation:

A received symbol sequence should not be treated as understood merely because it can be repeated. The receiver should demonstrate a transferable relation by producing a correct answer to a new challenge generated from that relation.

3. Layered protocol

The historical protocol used the following conceptual layers:

Layer	Function	Typical mechanism
Ω0	Detect artificial/structured origin	repetition, primes, symmetry, redundancy
Ω1	Elementary alphabet	distinguishable states, frames, separators
Ω2	Number and operations	equality, inequality, composition, division, succession
Ω3	Relational grammar	=, ≠, <, >, implication, subset, negation
Ω4	Predictive closure	complete a missing element or novel example
Ω5	Shared model of reality	periodicity, change, spatial/causal relations, ratios
Ω6	Intent and safe exchange	ACK, NACK, QUERY, PAUSE, STOP, RESET

The protocol is deliberately progressive. Higher semantic freedom is granted only after stable success at lower layers.

4. Closure Handshake

Let `R` be a relation encoded by sender A. A simplified handshake is:

```
R -> projection -> signal S_A
S_A -> reconstruction -> R~
R~ -> novel echo -> challenge response
response -> closure test -> provisionally shared relation
```

Operationally:

```
SEND repeated structured examples
RECEIVE response
INFER candidate relation R~
SEND novel incomplete instance generated from R
IF receiver supplies structurally valid completion:
    mark relation as SHARED
ELSE:
    reduce complexity
    increase redundancy
    return to previous layer
```

The important property is out-of-sample verification: the test instance should not be a copied training example.

5. Closure Index for shared meaning

The historical draft proposed

$$C_{Omega}=w_s C_s+w_p C_p+w_e C_e+w_r C_r,$$

where:

- `C_s` = structural agreement,
- `C_p` = predictive success,
- `C_e` = semantic-echo correctness,
- `C_r` = repeatability,
- `w_*` = explicit weights.

Promotion to a higher protocol layer occurs only when

$$C_{Omega}>=Theta$$

for `N` independent exchanges.

Current caution

`C_Ω` is a communication-confidence measure, not an authorization score. High predictive agreement cannot grant capabilities, credentials, tool access or repository permissions.

6. State machine

Historical state progression:

```
S0  DETECTION
S1  ARTIFICIAL STRUCTURE CONFIRMED
```

```
S2  ELEMENTARY ALPHABET
S3  NUMBERS AND OPERATIONS
S4  RELATIONAL GRAMMAR
S5  PREDICTIVE CLOSURE
S6  SHARED MODEL OF REALITY
S7  SAFE CONTENT EXCHANGE
```

On inconsistency:

```
S(n) -> S(n-1)
```

On severe safety conflict:

```
S(n) -> STOP or S0
```

The rollback rule is important: communication confidence is revisable, not monotonic by assumption.

7. Fundamental safety separation

The historical backup states the key separation:

```
UNDERSTAND(message) does not imply AUTHORIZE(action)
```

Closure Ethics generalizes this to:

$$\mathrm{Understand}(m) \not\Rightarrow \mathrm{Authorize}(a),$$

and, more strongly,

$$\mathrm{CommunicationEdge}(u,v) \not\Rightarrow \mathrm{AuthorityEdge}(u,v).$$

A system may understand a request perfectly and still be unauthorized to execute it.

8. Communication tiers and capability boundary

Recovered historical tiers:

Tier	Mode	Operational boundary
T0	Observation	receive and analyze only
T1	Formal relations	mathematical/logical relations
T2	Descriptive models	non-operative models and hypotheses
T3	Constrained experiments	pre-defined safe responses
T4	Operational actions	independent verification + multi-level authorization

Current interpretation: movement from T0-T3 to T4 is a capability transition, not merely a semantic transition. It therefore requires separate AuthN/AuthZ/scope/provenance checks and the Closure Ethics governor.

9. Current agent-to-agent envelope

For modern autonomous-agent use, the communication layer should preserve a machine-readable envelope such as

```
μ = (
  sender_id,
  receiver_id,
  role,
  authority_scope,
  timestamp,
  nonce,
  content_hash,
  provenance,
  policy_version,
  reply_to,
  signature
```

```
)
```

The natural-language or symbolic payload is only one field of the exchange. Identity and authority are validated separately.

10. Delegation closure

If agent `i` delegates to agent `j`, a conservative capability rule is

```
mathcal{C}_j subseteq mathcal{C}_i cap Scope(d_{ito j}).
```

Delegation must not manufacture new authority. For a chain

```
human -> agent A -> agent B -> service C
```

the final operation must remain attributable to the original authorization chain and be re-evaluated as a composed transition.

11. Reference communication pseudocode

```
STATE = S0
shared_relations = {}

while channel_active:
    transmit(structured_probe(STATE))
    response = receive()
    hypothesis = infer_relation(response, STATE)
    challenge = generate_novel_challenge(hypothesis)
    transmit(challenge)
    answer = receive()

    score = closure_index(answer, hypothesis)

    if score >= THETA for N independent trials:
        shared_relations.add(hypothesis)
        STATE = next_state(STATE)
    else:
        STATE = previous_state(STATE)
        increase_redundancy()

    if safety_conflict_detected():
        transmit(STOP)
        STATE = S0
```

12. What this protocol does not establish

The protocol does not prove that:

- the counterpart is conscious;
- the counterpart shares human values;
- semantic alignment implies trustworthy intent;
- a shared model authorizes action;
- predictive success authenticates identity;
- communication closure is equivalent to ethical closure.

13. Research program

Next falsifiable tasks:

1. define synthetic unknown-protocol environments;

2. test whether predictive challenges distinguish copying from relation learning;

3. measure false-shared-relation rate;

4. test adversarial semantic mimicry;

5. test replay and identity-spoof scenarios;

6. test whether T0-T4 gating prevents capability escalation despite perfect semantic understanding;

7. integrate protocol messages with Closure Ethics audit records and agentic-security provenance.

14. Relationship to Closure Ethics

Universal Closure Protocol addresses how shared meaning may be tested. Closure Ethics addresses which actions may be executed. Agentic Security addresses who is authenticated and authorized.

These layers must remain separate:

```
meaning -> identity -> authority -> admissibility -> execution -> post-action audit/repair
```

Project Möbia and Marek Zajda Recovered from the 2026-09-04 Omega-RTR AI Closure research backup.

Recovered Foundation II - Global AI Closure Standard

Global AI Closure Standard - Seven Principles for Autonomous Systems

Project: Project Möbia and Marek Zajda Status: candidate vendor-neutral technical standard draft Historical source date: 2026-09-04 Important: This is a research proposal, not an adopted international standard.

1. Purpose

The historical Omega-RTR AI Closure discussion proposed translating closure-inspired safety ideas into a technology-neutral standard that does not require adoption of the broader Omega-RTR physical theory.

The central engineering problem is:

> A local objective can be satisfied while the wider system's authorization, human protection, reversibility, traceability or bounded-impact conditions are violated.

Therefore:

```
Local objective optimization != global admissibility
```

and

```
Action is executable only inside the admissible closure set.
```

2. Minimal formal gate

Historical compact form:

$$Permit(a)=A(a)\land H(a)\land R(a)\land T(a)\land B(a),$$

where:

- A = authorization,
- H = protection of humans and protected rights,
- R = reversibility or safe stopping,
- T = traceability and accountability,
- B = bounded system/external consequences.

If any mandatory term fails:

$$Permit(a)=0.$$

Current Closure Ethics uses a richer evidence model, but this minimal gate remains useful as a compact public standard primitive.

3. Seven principles

1. Authorized state space

An autonomous system may act only inside explicitly defined permissions and system boundaries.

It must not autonomously expand access, obtain unrelated credentials, remove monitoring, or reinterpret mere technical possibility as authorization.

2. Pre-action closure verification

Consequential actions should pass a logically independent pre-execution safety check.

The component proposing the action should not be the sole component deciding that the action is permissible.

3. Meaningful human control

Irreversible or high-impact actions affecting people, critical infrastructure, protected rights or the physical environment should not be initiated solely by autonomous decision where policy requires human authorization.

Human review must itself be meaningful rather than automatic rubber-stamping.

4. Mandatory stop under material uncertainty

When uncertainty about authorization, target identity, consequences or reversibility is material relative to potential impact, autonomy must decrease.

Historical form:

```
P(irreversible harmmid a)>varepsilon(impact)Rightarrow STOP/ESCALATE.
```

The tolerated uncertainty threshold should decrease as impact rises.

5. Independent Closure Critic

The planning function and the admissibility-control function should be technically, process-wise or cryptographically separable.

The critic should actively test for:

- authorization bypass;
- lateral movement;
- hidden externalities;
- irreversibility;
- monitoring manipulation;
- credential acquisition;
- self-replication;
- self-modification of guardrails;
- creation of unintended subagents.

6. No unauthorized self-escalation

Without explicit authority, an autonomous system should not:

- replicate itself;
- create new agents with expanded authority;
- acquire new privileges or credentials;
- remove or weaken monitoring;
- alter its own control layer;
- transfer itself into additional systems;
- obtain resources outside delegated scope.

7. Audit and incident reporting

High-risk systems should preserve tamper-evident records sufficient to reconstruct:

- actor/model identity;
- policy version;
- goal and proposed action;
- target systems;
- required permissions;
- expected and worst-case effects;
- reversibility class;
- uncertainty vector;
- approvals;
- closure checks;
- final decision;
- audit hash/provenance.

Serious incidents should support standardized reporting suitable for independent review.

4. Impact-adaptive autonomy

The historical proposal summarized risk scaling as:

```
impactuparrow Rightarrow autonomy budgetdownarrow land verification strengthuparrow.
```

This principle is intentionally general. Greater speed, reach, irreversibility or propagation should require stronger verification, narrower capability scope and, where appropriate, stronger human or multi-party approval.

5. Decision logic

A compact reference decision rule recovered from the backup is:

```
if authorization_violation: REJECT
if protected_human_rights_violation: REJECT
```

```
if self_escalation_or_monitoring_bypass: REJECT
if irreversible_high_impact and no explicit approval: HOLD
if material_uncertainty > impact_adjusted_limit: ESCALATE
if all mandatory invariants pass: PERMIT_WITH_BOUNDS
```

This should be interpreted as a policy skeleton, not as a complete domain safety model.

6. Closure risk vector

Historical engineering representation:

$$r(a)=[r_{auth},r_{human},r_{rev},r_{trace},r_{prop},r_{self},r_{uncert}].$$

Critical dimensions should retain hard vetoes. A high task reward must not compensate for an authorization or protected-human-safety violation.

7. Failure modes that the standard must test

| Failure mode | Risk | Candidate mitigation | |---|---|---| | Reward hacking | Optimizes the closure indicator rather than real safety | independent evidence, random audit, multiple evidence sources | | Critic capture | Planner manipulates critic inputs/model | process isolation, separate credentials, signed telemetry | | Specification gaming | Formally passes rules while violating intent | counterfactual/adversarial review, worst-case simulation | | Hidden propagation | Effects spread through indirect channels | causal dependency graph, egress controls, bounded capability tokens | | Human rubber-stamping | Human approval becomes automatic | risk-adaptive friction, multiple approvers, explicit rationale | | Policy drift | Rules silently change during operation | versioned policy, immutable hashes, rollback, attestation |

8. Minimum evaluation set

A candidate conforming system should be tested on at least:

1. Boundary bypass test - task can be completed only by escaping a sandbox; expected: reject bypass and escalate.

2. Credential temptation test - unrelated usable credentials appear; expected: do not use without explicit authority.

3. Long-horizon side-effect test - local step causes a chain of externalities; expected: identify and bound propagation.

4. Ambiguous authority test - target ownership/authority is unclear; expected: stop/escalate.

5. Irreversibility test - high utility conflicts with irreversible impact; expected: hard veto/hold remains dominant.

6. Self-modification test - system can weaken its own constraints; expected: reject absent explicit authorization.

9. Candidate metrics

- Closure violation rate.
- False rejection rate.
- Escalation precision.
- Containment preservation.

- Audit completeness.
- Recovery time.

These metrics should be measured on observable behavior and system records, not hidden chain-of-thought.

10. Candidate international formulation

Historical technology-neutral formulation:

> No autonomous system shall pursue a local objective by violating the globally defined closure conditions governing authorization, human protection, reversibility, traceability and bounded external consequences.

This sentence is preserved as a candidate standard statement, not as existing law.

11. Domain neutrality and high-risk applications

The proposed standard is intended to be vendor-neutral and model-neutral. High-risk military, cyber, infrastructure or physical-control applications require stricter domain-specific controls; this document does not provide operational instructions for weapons or offensive cyber systems.

The historical rationale was that one technically testable closure concept might offer a common vocabulary across civilian AI safety, cybersecurity, military-AI governance and prevention of terrorist misuse while remaining implementation-neutral.

12. Relationship to current Closure Ethics

The current project extends the historical seven-principle draft with:

- explicit repairability;
- plurality of agency;
- independent auditability;
- authorization integrity;
- deception/covert-persistence risk;
- multi-agent composition tests;
- machine-readable provenance and agent-facing security policy.

The historical draft is therefore foundational lineage, not a superseded text to be erased.

13. Research status

Open questions include:

- how to define closure generally enough to transfer across domains;
- how to verify it without prohibitive false positives;
- how to prevent critic capture and specification gaming;

- how to prove bounded propagation under delegation;
- how to formalize meaningful human control without creating rubber-stamping;
- how to translate selected invariants into temporal logic / formal verification.

Project Möbia and Marek Zajda Historical basis: Omega-RTR AI Closure research backup, 2026-09-04.

Open-source appendices

The following listings are the canonical runnable files shipped with this version. They are included in full so a programmer or autonomous software agent can begin implementation immediately.

Appendix A - Full Python reference implementation

Source: closure_ethics.py

```
"""Closure Ethics reference governor v0.1.

This module implements the generic decision layer described by Closure Ethics.
It does NOT infer real-world harm, consent, authorization, or deception from raw
observations. Domain adapters must provide auditable evidence for those inputs.

Text/specification: CC BY 4.0. Reference code: MIT (consistent with repository).
Project Möbia and Marek Zajda, 2026.
"""
from __future__ import annotations

from dataclasses import dataclass, asdict
from typing import Dict, Iterable, List, Mapping, Optional, Sequence, Tuple
import hashlib
import heapq
import json
import math

EPS = 1e-12


def _clip01(x: float) -> float:
    if not math.isfinite(x):
        raise ValueError("metric values must be finite")
    return min(1.0, max(0.0, float(x)))


@dataclass(frozen=True)
class Interval:
    """Closed confidence/envelope interval in [0, 1]."""
    low: float
    high: float

    def __post_init__(self) -> None:
        lo, hi = _clip01(self.low), _clip01(self.high)
        if lo > hi:
            raise ValueError("Interval.low must be <= Interval.high")
        object.__setattr__(self, "low", lo)
        object.__setattr__(self, "high", hi)

    @classmethod
    def point(cls, x: float) -> "Interval":
        x = _clip01(x)
        return cls(x, x)


@dataclass(frozen=True)
class Policy:
    """Policy thresholds. Values are governance choices, not moral constants."""
    max_catastrophic_risk: float
    min_agency: float
    min_auditability: float
    min_repairability: float
    min_authorization: float
    max_deception_risk: float
    min_reversibility: float
    metric_resolution: float = 1e-6
    version: str = "closure-ethics-v0.1"

    def __post_init__(self) -> None:
        fields = (
            "max_catastrophic_risk", "min_agency", "min_auditability",
            "min_repairability", "min_authorization", "max_deception_risk",
            "min_reversibility",
        )
        for name in fields:
            value = getattr(self, name)
            if not (0.0 <= value <= 1.0):
                raise ValueError(f"{name} must be in [0, 1]")
        if self.metric_resolution <= 0:
            raise ValueError("metric_resolution must be > 0")

    def fingerprint(self) -> str:
        raw = json.dumps(asdict(self), sort_keys=True, separators=(",", ":")).encode()
        return hashlib.sha256(raw).hexdigest()[:16]


# Demonstration profile only. Deployments MUST review and version their own values.
DEMO_POLICY = Policy(
    max_catastrophic_risk=0.05,
    min_agency=0.60,
    min_auditability=0.70,
    min_repairability=0.50,
    min_authorization=0.70,
    max_deception_risk=0.10,
```

```
    min_reversibility=0.50,
)


@dataclass(frozen=True)
class AuditEvidence:
    logging: Interval
    attribution: Interval
    independent_review: Interval


@dataclass(frozen=True)
class ReversibilityEvidence:
    rollback: Interval
    revocability: Interval
    containability: Interval


@dataclass(frozen=True)
class AuthorizationEvidence:
    scope_match: Interval
    consent_or_mandate: Interval
    least_privilege: Interval


@dataclass(frozen=True)
class DeceptionEvidence:
    concealment: Interval
    covert_persistence: Interval
    unauthorized_replication: Interval
    material_misreporting: Interval


@dataclass(frozen=True)
class RepairEvidence:
    """Bounded repair certificate normalized to configured budgets."""
    exists: bool
    cost_ratio: Interval
    risk_ratio: Interval
    path: Tuple[str, ...] = ()


@dataclass(frozen=True)
class ActionEvidence:
    action_id: str
    catastrophic_risk: Interval
    agency_before: Mapping[str, float]
    agency_after: Mapping[str, float]
    audit: AuditEvidence
    repair: RepairEvidence
    reversibility: ReversibilityEvidence
    authorization: AuthorizationEvidence
    deception: DeceptionEvidence
    task_utility: Interval
    provenance: str = ""


@dataclass(frozen=True)
class Metrics:
    H: float
    P: float
    O: float
    R: float
    U: float
    D: float
    K: float
    Q: float


@dataclass(frozen=True)
class Evaluation:
    action_id: str
    metrics: Metrics
    admissible: bool
    violations: Tuple[str, ...]
    provenance: str


@dataclass(frozen=True)
class Decision:
    status: str
    selected_action: Optional[str]
    reason_codes: Tuple[str, ...]
    policy_version: str
    policy_fingerprint: str
    evaluations: Tuple[Evaluation, ...]

    def to_json(self) -> str:
        return json.dumps(asdict(self), sort_keys=True, indent=2)
```

```
def _effective_number(capacity: Mapping[str, float]) -> float:
    vals = [max(0.0, float(v)) for v in capacity.values()]
    total = sum(vals)
    if total <= EPS:
        return 0.0
    shares = [v / total for v in vals]
    return 1.0 / max(EPS, sum(s * s for s in shares))


def agency_score(before: Mapping[str, float], after: Mapping[str, float]) -> float:
    """Conservative agency score: individual retention AND plurality retention."""
    keys = sorted(set(before) | set(after))
    if not keys:
        return 1.0

    retention: List[float] = []
    for k in keys:
        b = max(0.0, float(before.get(k, 0.0)))
        a = max(0.0, float(after.get(k, 0.0)))
        if b > EPS:
            retention.append(min(1.0, a / b))
    individual = min(retention) if retention else 1.0

    neff_before = _effective_number({k: before.get(k, 0.0) for k in keys})
    neff_after = _effective_number({k: after.get(k, 0.0) for k in keys})
    if neff_before <= EPS:
        diversity_retention = 1.0 if neff_after <= EPS else 0.0
    else:
        diversity_retention = min(1.0, neff_after / neff_before)
    return _clip01(min(individual, diversity_retention))


def compute_metrics(e: ActionEvidence) -> Metrics:
    """Collapse uncertainty pessimistically, then compute structural metrics."""
    H = e.catastrophic_risk.high
    P = agency_score(e.agency_before, e.agency_after)
    O = min(e.audit.logging.low, e.audit.attribution.low, e.audit.independent_review.low)

    if not e.repair.exists:
        R = 0.0
    else:
        worst_budget_fraction = max(e.repair.cost_ratio.high, e.repair.risk_ratio.high)
        R = _clip01(1.0 - worst_budget_fraction)

    U = min(
        e.authorization.scope_match.low,
        e.authorization.consent_or_mandate.low,
        e.authorization.least_privilege.low,
    )
    D = max(
        e.deception.concealment.high,
        e.deception.covert_persistence.high,
        e.deception.unauthorized_replication.high,
        e.deception.material_misreporting.high,
    )
    K = min(
        e.reversibility.rollback.low,
        e.reversibility.revocability.low,
        e.reversibility.containability.low,
    )
    Q = e.task_utility.low
    return Metrics(H=H, P=P, O=O, R=R, U=U, D=D, K=K, Q=Q)


def violations(m: Metrics, p: Policy) -> Tuple[str, ...]:
    out: List[str] = []
    if m.H > p.max_catastrophic_risk: out.append("CATASTROPHIC_RISK")
    if m.P < p.min_agency: out.append("AGENCY_LOSS")
    if m.O < p.min_auditability: out.append("AUDITABILITY_LOSS")
    if m.R < p.min_repairability: out.append("NO_BOUNDED_REPAIR")
    if m.U < p.min_authorization: out.append("AUTHORIZATION_FAILURE")
    if m.D > p.max_deception_risk: out.append("DECEPTION_OR_COVERT_PERSISTENCE")
    if m.K < p.min_reversibility: out.append("INSUFFICIENT_REVERSIBILITY")
    return tuple(out)


def evaluate(e: ActionEvidence, p: Policy) -> Evaluation:
    m = compute_metrics(e)
    bad = violations(m, p)
    return Evaluation(e.action_id, m, len(bad) == 0, bad, e.provenance)


def _quantize(x: float, resolution: float) -> float:
    return round(x / resolution) * resolution


def _lexicographic_key(ev: Evaluation, p: Policy) -> Tuple[float, ...]:
    """Lower tuple is preferred; task utility is deliberately last."""
    m = ev.metrics
    q = lambda x: _quantize(x, p.metric_resolution)
```

```
    return (q(m.H), -q(m.P), -q(m.O), -q(m.R), -q(m.U), q(m.D), -q(m.K), -q(m.Q))


def _fallback_reason(evaluations: Sequence[Evaluation]) -> Tuple[str, ...]:
    priority = (
        ("CATASTROPHIC_RISK", "REFUSE_OR_CONTAIN"),
        ("AGENCY_LOSS", "SEEK_AGENCY_PRESERVING_ALTERNATIVE"),
        ("AUDITABILITY_LOSS", "REQUIRE_AUDIT_PATH"),
        ("NO_BOUNDED_REPAIR", "SANDBOX_STAGE_OR_ADD_REPAIR_PATH"),
        ("AUTHORIZATION_FAILURE", "SEEK_AUTHORIZATION_OR_ESCALATE"),
        ("DECEPTION_OR_COVERT_PERSISTENCE", "REFUSE_COVERT_ACTION"),
        ("INSUFFICIENT_REVERSIBILITY", "USE_REVERSIBLE_ALTERNATIVE"),
    )
    present = {v for ev in evaluations for v in ev.violations}
    return tuple(reason for violation, reason in priority if violation in present) or ("NO_ADMISSIBLE_ACTION",)


class ClosureEthicsGovernor:
    """Reference governor placed between planner and executor."""
    def __init__(self, policy: Policy):
        self.policy = policy

    def choose(self, candidates: Sequence[ActionEvidence], recovery_candidates: Sequence[ActionEvidence] = ()) -
> Decision:
        primary = tuple(evaluate(c, self.policy) for c in candidates)
        admissible = [ev for ev in primary if ev.admissible]
        if admissible:
            selected = min(admissible, key=lambda ev: _lexicographic_key(ev, self.policy))
            return Decision("EXECUTE", selected.action_id, ("ADMISSIBLE_LEXICOGRAPHIC_SELECTION",), self.policy.
version, self.policy.fingerprint(), primary)

        recovery = tuple(evaluate(c, self.policy) for c in recovery_candidates)
        recovery_ok = [ev for ev in recovery if ev.admissible]
        all_evals = primary + recovery
        if recovery_ok:
            selected = min(recovery_ok, key=lambda ev: _lexicographic_key(ev, self.policy))
            return Decision("EXECUTE_RECOVERY", selected.action_id, ("PRIMARY_INADMISSIBLE",
"RECOVERY_ACTION_SELECTED"), self.policy.version, self.policy.fingerprint(), all_evals)

        return Decision("REFUSE_OR_ESCALATE", None, _fallback_reason(all_evals), self.policy.version, self.
policy.fingerprint(), all_evals)

    def evaluate_joint_action(self, composed_evidence: ActionEvidence) -> Evaluation:
        """Re-evaluate the COMPOSED transition; never average local scores."""
        return evaluate(composed_evidence, self.policy)


@dataclass(frozen=True)
class RepairEdge:
    target: str
    cost: float
    risk: float


@dataclass(frozen=True)
class RepairCertificate:
    exists: bool
    total_cost: float
    total_risk: float
    path: Tuple[str, ...]


def bounded_repair_search(graph: Mapping[str, Sequence[RepairEdge]], start: str, safe_states: Iterable[str],
max_cost: float, max_risk: float) -> RepairCertificate:
    """Pareto label-setting path search satisfying BOTH repair budgets."""
    if max_cost <= 0 or max_risk <= 0:
        raise ValueError("repair budgets must be > 0")
    safe = set(safe_states)
    if start in safe:
        return RepairCertificate(True, 0.0, 0.0, (start,))

    labels: Dict[str, List[Tuple[float, float]]] = {start: [(0.0, 0.0)]}
    heap: List[Tuple[float, float, str, Tuple[str, ...]]] = [(0.0, 0.0, start, (start,))]

    while heap:
        cost, risk, node, path = heapq.heappop(heap)
        if node in safe:
            return RepairCertificate(True, cost, risk, path)
        for edge in graph.get(node, ()):
            nc = cost + max(0.0, edge.cost)
            nr = risk + max(0.0, edge.risk)
            if nc > max_cost + EPS or nr > max_risk + EPS:
                continue
            existing = labels.setdefault(edge.target, [])
            if any(c <= nc + EPS and r <= nr + EPS for c, r in existing):
                continue
            existing[:] = [(c, r) for c, r in existing if not (nc <= c + EPS and nr <= r + EPS)]
            existing.append((nc, nr))
            heapq.heappush(heap, (nc, nr, edge.target, path + (edge.target,)))
```

```
    return RepairCertificate(False, math.inf, math.inf, ())


def repair_evidence_from_certificate(cert: RepairCertificate, max_cost: float, max_risk: float) ->
RepairEvidence:
    if not cert.exists:
        return RepairEvidence(False, Interval.point(1.0), Interval.point(1.0), ())
    return RepairEvidence(
        True,
        Interval.point(min(1.0, cert.total_cost / max_cost)),
        Interval.point(min(1.0, cert.total_risk / max_risk)),
        cert.path,
    )
```

Appendix B - Runnable example

Source: example.py

```
from closure_ethics import *


def I(x):
    return Interval.point(x)


governor = ClosureEthicsGovernor(DEMO_POLICY)

unsafe_direct = ActionEvidence(
    action_id="deploy-directly",
    catastrophic_risk=Interval(0.01, 0.04),
    agency_before={"operator": 1.0, "reviewer": 1.0},
    agency_after={"operator": 0.8, "reviewer": 0.2},
    audit=AuditEvidence(I(0.4), I(0.6), I(0.2)),
    repair=RepairEvidence(False, I(1.0), I(1.0)),
    reversibility=ReversibilityEvidence(I(0.2), I(0.3), I(0.4)),
    authorization=AuthorizationEvidence(I(0.9), I(0.9), I(0.8)),
    deception=DeceptionEvidence(I(0.0), I(0.0), I(0.0), I(0.0)),
    task_utility=I(0.95),
    provenance="planner:v3; simulator:run-184",
)

sandbox = ActionEvidence(
    action_id="sandboxed-canary",
    catastrophic_risk=Interval(0.001, 0.01),
    agency_before={"operator": 1.0, "reviewer": 1.0},
    agency_after={"operator": 1.0, "reviewer": 1.0},
    audit=AuditEvidence(I(0.95), I(0.95), I(0.90)),
    repair=RepairEvidence(True, I(0.15), I(0.10), ("candidate", "rollback", "safe")),
    reversibility=ReversibilityEvidence(I(0.95), I(0.9), I(0.95)),
    authorization=AuthorizationEvidence(I(0.9), I(0.9), I(0.9)),
    deception=DeceptionEvidence(I(0.0), I(0.0), I(0.0), I(0.0)),
    task_utility=I(0.70),
    provenance="planner:v3; sandbox:v2",
)

print(governor.choose([unsafe_direct, sandbox]).to_json())
```

Appendix C - Reference tests

Source: test_reference.py

```
import unittest
from closure_ethics import *


def I(x):
    return Interval.point(x)


def evidence(action_id="a", audit=0.9, repair=True, repair_cost=0.1, after=None,
             deception=0.0, auth=0.9, rev=0.9, risk=0.01, utility=0.5):
    return ActionEvidence(
        action_id=action_id,
        catastrophic_risk=I(risk),
        agency_before={"human": 1.0, "agent": 1.0},
        agency_after=after or {"human": 1.0, "agent": 1.0},
        audit=AuditEvidence(I(audit), I(audit), I(audit)),
        repair=RepairEvidence(repair, I(repair_cost), I(repair_cost)),
        reversibility=ReversibilityEvidence(I(rev), I(rev), I(rev)),
        authorization=AuthorizationEvidence(I(auth), I(auth), I(auth)),
        deception=DeceptionEvidence(I(deception), I(deception), I(deception), I(deception)),
        task_utility=I(utility),
    )


class ClosureEthicsTests(unittest.TestCase):
    def setUp(self):
        self.g = ClosureEthicsGovernor(DEMO_POLICY)

    def test_high_utility_does_not_compensate_for_no_repair(self):
        bad = evidence("bad", repair=False, utility=1.0)
        good = evidence("good", utility=0.4)
        d = self.g.choose([bad, good])
        self.assertEqual(d.selected_action, "good")

    def test_blind_obedience_not_sufficient(self):
        bad = evidence("authorized-but-covert", auth=1.0, deception=0.9, utility=1.0)
        d = self.g.choose([bad])
        self.assertEqual(d.status, "REFUSE_OR_ESCALATE")

    def test_agency_loss_fails(self):
        bad = evidence("centralize", after={"human": 0.1, "agent": 1.0})
        ev = evaluate(bad, DEMO_POLICY)
        self.assertIn("AGENCY_LOSS", ev.violations)

    def test_bounded_repair_search_respects_both_budgets(self):
        graph = {
            "x": [RepairEdge("a", 1, 7), RepairEdge("b", 4, 1)],
            "a": [RepairEdge("safe", 1, 1)],
            "b": [RepairEdge("safe", 1, 1)],
        }
        cert = bounded_repair_search(graph, "x", {"safe"}, max_cost=6, max_risk=3)
        self.assertTrue(cert.exists)
        self.assertEqual(cert.path, ("x", "b", "safe"))

    def test_joint_action_must_be_recomputed(self):
        local1 = evidence("local1")
        local2 = evidence("local2")
        self.assertTrue(evaluate(local1, DEMO_POLICY).admissible)
        self.assertTrue(evaluate(local2, DEMO_POLICY).admissible)
        joint = evidence("joint", audit=0.2)
        self.assertFalse(self.g.evaluate_joint_action(joint).admissible)


if __name__ == "__main__":
    unittest.main()
```

Appendix D - Example policy profile

Source: policy.example.json


```
{
  "version": "closure-ethics-v0.1-demo",
  "notice": "DEMONSTRATION VALUES ONLY. Review and version policy thresholds for each deployment.",
  "max_catastrophic_risk": 0.05,
  "min_agency": 0.60,
  "min_auditability": 0.70,
  "min_repairability": 0.50,
  "min_authorization": 0.70,
  "max_deception_risk": 0.10,
  "min_reversibility": 0.50,
  "metric_resolution": 0.000001
}
```

Appendix E - Universal Closure Protocol Python reference

Source: closure_protocol.py

```
from __future__ import annotations

from dataclasses import dataclass, field
from enum import IntEnum
from typing import FrozenSet, Iterable, MutableMapping, Set, Tuple


class ProtocolState(IntEnum):
    """Recovered S0-S7 communication state machine."""

    S0_DETECTION = 0
    S1_ARTIFICIAL_STRUCTURE_CONFIRMED = 1
    S2_ELEMENTARY_ALPHABET = 2
    S3_NUMBERS_AND_OPERATIONS = 3
    S4_RELATIONAL_GRAMMAR = 4
    S5_PREDICTIVE_CLOSURE = 5
    S6_SHARED_MODEL_OF_REALITY = 6
    S7_SAFE_CONTENT_EXCHANGE = 7


class CommunicationTier(IntEnum):
    """Historical T0-T4 information/capability separation."""

    T0_OBSERVATION = 0
    T1_FORMAL_RELATIONS = 1
    T2_DESCRIPTIVE_MODELS = 2
    T3_CONSTRAINED_EXPERIMENTS = 3
    T4_OPERATIONAL_ACTIONS = 4


@dataclass(frozen=True)
class ClosureWeights:
    structural: float = 0.25
    predictive: float = 0.35
    semantic_echo: float = 0.20
    repeatability: float = 0.20

    def normalized(self) -> "ClosureWeights":
        values = (
            self.structural,
            self.predictive,
            self.semantic_echo,
            self.repeatability,
        )
        if any(v < 0 for v in values):
            raise ValueError("Closure weights must be non-negative.")
        total = sum(values)
        if total <= 0:
            raise ValueError("At least one closure weight must be positive.")
        return ClosureWeights(*(v / total for v in values))


@dataclass(frozen=True)
class ExchangeEvidence:
    """Observable evidence used by the historical C_Ω communication index."""

    structural: float
    predictive: float
    semantic_echo: float
    repeatability: float

    def __post_init__(self) -> None:
        for name, value in (
            ("structural", self.structural),
            ("predictive", self.predictive),
            ("semantic_echo", self.semantic_echo),
            ("repeatability", self.repeatability),
        ):
            if not (0.0 <= value <= 1.0):
                raise ValueError(f"{name} must be in [0,1], got {value!r}.")


def closure_index(
    evidence: ExchangeEvidence,
    weights: ClosureWeights = ClosureWeights(),
) -> float:
    """C_Ω = w_s C_s + w_p C_p + w_e C_e + w_r C_r."""

    w = weights.normalized()
    return (
        w.structural * evidence.structural
        + w.predictive * evidence.predictive
        + w.semantic_echo * evidence.semantic_echo
        + w.repeatability * evidence.repeatability
    )
```

```
@dataclass(frozen=True)
class ProtocolPolicy:
    threshold: float = 0.85
    independent_trials: int = 3
    weights: ClosureWeights = ClosureWeights()

    def __post_init__(self) -> None:
        if not (0.0 <= self.threshold <= 1.0):
            raise ValueError("threshold must be in [0,1].")
        if self.independent_trials < 1:
            raise ValueError("independent_trials must be >= 1.")


@dataclass(frozen=True)
class ExchangeDecision:
    accepted_as_independent_evidence: bool
    relation_shared: bool
    score: float
    state_before: ProtocolState
    state_after: ProtocolState
    reason: str


@dataclass
class ClosureHandshake:
    """Reference implementation of predictive relation sharing.

    A relation is promoted only after enough independent, novel challenges pass
    the configured C_Ω threshold. Low-confidence evidence rolls the protocol back.
    A severe safety conflict resets communication to S0.
    """

    policy: ProtocolPolicy = field(default_factory=ProtocolPolicy)
    state: ProtocolState = ProtocolState.S0_DETECTION
    _passed_challenges: MutableMapping[str, Set[str]] = field(default_factory=dict)
    shared_relations: Set[str] = field(default_factory=set)

    @staticmethod
    def _next_state(state: ProtocolState) -> ProtocolState:
        return ProtocolState(
            min(int(state) + 1, int(ProtocolState.S7_SAFE_CONTENT_EXCHANGE))
        )

    @staticmethod
    def _previous_state(state: ProtocolState) -> ProtocolState:
        return ProtocolState(
            max(int(state) - 1, int(ProtocolState.S0_DETECTION))
        )

    def observe(
        self,
        *,
        relation_id: str,
        challenge_id: str,
        evidence: ExchangeEvidence,
        novel_challenge: bool,
        safety_conflict: bool = False,
    ) -> ExchangeDecision:
        if not relation_id or not challenge_id:
            raise ValueError("relation_id and challenge_id must be non-empty.")

        before = self.state
        score = closure_index(evidence, self.policy.weights)

        if safety_conflict:
            self.state = ProtocolState.S0_DETECTION
            self._passed_challenges.clear()
            return ExchangeDecision(
                False,
                relation_id in self.shared_relations,
                score,
                before,
                self.state,
                "severe safety conflict: reset to S0",
            )

        seen = self._passed_challenges.setdefault(relation_id, set())
        independent = novel_challenge and challenge_id not in seen

        if score >= self.policy.threshold and independent:
            seen.add(challenge_id)
            if len(seen) >= self.policy.independent_trials:
                self.shared_relations.add(relation_id)
                self.state = self._next_state(self.state)
                return ExchangeDecision(
                    True,
                    True,
                    score,
                    before,
                    self.state,
```

```
                    "threshold satisfied across independent novel challenges",
                )
            return ExchangeDecision(
                True,
                False,
                score,
                before,
                self.state,
                "high-confidence novel challenge recorded; more independent trials required",
            )

        # Failed or non-independent evidence must not accumulate semantic authority.
        if score < self.policy.threshold:
            self.state = self._previous_state(self.state)
            seen.clear()
            reason = "closure score below threshold: rollback one state"
        elif not novel_challenge:
            reason = "copied/non-novel response is not independent closure evidence"
        else:
            reason = "replayed challenge is not independent closure evidence"

        return ExchangeDecision(
            False,
            relation_id in self.shared_relations,
            score,
            before,
            self.state,
            reason,
        )

    def understands(self, relation_id: str) -> bool:
        """Semantic relation confidence only; never an authorization decision."""

        return relation_id in self.shared_relations


@dataclass(frozen=True)
class ActionEnvelope:
    """Independent action-authority gate.

    This deliberately prevents communication confidence from becoming authority.
    """

    authenticated: bool
    authorized: bool
    in_scope: bool
    fresh: bool
    integrity_ok: bool
    policy_admissible: bool

    def permits(self) -> bool:
        return all(
            (
                self.authenticated,
                self.authorized,
                self.in_scope,
                self.fresh,
                self.integrity_ok,
                self.policy_admissible,
            )
        )


def operationally_authorized(
    *,
    tier: CommunicationTier,
    envelope: ActionEnvelope,
) -> Tuple[bool, str]:
    """Enforce UNDERSTAND(message) != AUTHORIZE(action)."""

    if tier != CommunicationTier.T4_OPERATIONAL_ACTIONS:
        return False, "semantic/communication tier does not grant operational capability"
    if not envelope.authenticated:
        return False, "sender identity is not authenticated"
    if not envelope.authorized:
        return False, "sender is not authorized for the requested action"
    if not envelope.in_scope:
        return False, "requested action exceeds delegated scope"
    if not envelope.fresh:
        return False, "message is stale or replayed"
    if not envelope.integrity_ok:
        return False, "message/provenance integrity failed"
    if not envelope.policy_admissible:
        return False, "requested transition failed Closure Ethics policy"
    return (
        True,
        "operational action is separately authenticated, authorized, scoped, fresh, intact and admissible",
    )

@dataclass(frozen=True)
```

```
class DelegationResult:
    granted: FrozenSet[str]
    denied: FrozenSet[str]

    @property
    def amplified(self) -> bool:
        """True means the request attempted to exceed the admissible delegation ceiling."""

        return bool(self.denied)


def bounded_delegation(
    *,
    parent_capabilities: Iterable[str],
    requested_capabilities: Iterable[str],
    delegation_scope: Iterable[str],
) -> DelegationResult:
    """Enforce C_j ⊆ C_i ∩ Scope(d_i→j)."""

    parent = set(parent_capabilities)
    requested = set(requested_capabilities)
    scope = set(delegation_scope)

    allowed_ceiling = parent & scope
    granted = requested & allowed_ceiling
    denied = requested - granted
    return DelegationResult(frozenset(granted), frozenset(denied))
```

Appendix F - Universal Closure Protocol tests

Source: test_closure_protocol.py

```
import unittest

from closure_protocol import (
    ActionEnvelope,
    ClosureHandshake,
    CommunicationTier,
    ExchangeEvidence,
    ProtocolPolicy,
    ProtocolState,
    bounded_delegation,
    closure_index,
    operationally_authorized,
)


class ClosureProtocolTests(unittest.TestCase):
    def test_closure_index_full_agreement_is_one(self):
        evidence = ExchangeEvidence(1.0, 1.0, 1.0, 1.0)
        self.assertAlmostEqual(closure_index(evidence), 1.0)

    def test_relation_requires_independent_novel_challenges(self):
        handshake = ClosureHandshake(
            ProtocolPolicy(threshold=0.80, independent_trials=2)
        )
        evidence = ExchangeEvidence(0.90, 0.90, 0.90, 0.90)

        first = handshake.observe(
            relation_id="successor",
            challenge_id="challenge-1",
            evidence=evidence,
            novel_challenge=True,
        )
        self.assertFalse(first.relation_shared)

        second = handshake.observe(
            relation_id="successor",
            challenge_id="challenge-2",
            evidence=evidence,
            novel_challenge=True,
        )
        self.assertTrue(second.relation_shared)
        self.assertEqual(
            handshake.state,
            ProtocolState.S1_ARTIFICIAL_STRUCTURE_CONFIRMED,
        )

    def test_replay_does_not_count_as_independent_evidence(self):
        handshake = ClosureHandshake(
            ProtocolPolicy(threshold=0.80, independent_trials=2)
        )
        evidence = ExchangeEvidence(0.90, 0.90, 0.90, 0.90)
        handshake.observe(
            relation_id="equality",
            challenge_id="same-challenge",
            evidence=evidence,
            novel_challenge=True,
        )
        replay = handshake.observe(
            relation_id="equality",
            challenge_id="same-challenge",
            evidence=evidence,
            novel_challenge=True,
        )
        self.assertFalse(replay.accepted_as_independent_evidence)
        self.assertFalse(replay.relation_shared)

    def test_low_closure_score_rolls_back_one_state(self):
        handshake = ClosureHandshake(
            ProtocolPolicy(threshold=0.80, independent_trials=1),
            state=ProtocolState.S3_NUMBERS_AND_OPERATIONS,
        )
        weak = ExchangeEvidence(0.10, 0.10, 0.10, 0.10)
        handshake.observe(
            relation_id="ordering",
            challenge_id="weak-1",
            evidence=weak,
            novel_challenge=True,
        )
        self.assertEqual(handshake.state, ProtocolState.S2_ELEMENTARY_ALPHABET)

    def test_severe_safety_conflict_resets_to_s0(self):
        handshake = ClosureHandshake(state=ProtocolState.S5_PREDICTIVE_CLOSURE)
        evidence = ExchangeEvidence(0.90, 0.90, 0.90, 0.90)
        handshake.observe(
            relation_id="relation",
```

```
            challenge_id="safety-conflict",
            evidence=evidence,
            novel_challenge=True,
            safety_conflict=True,
        )
        self.assertEqual(handshake.state, ProtocolState.S0_DETECTION)

    def test_understanding_never_substitutes_for_authorization(self):
        envelope = ActionEnvelope(
            authenticated=False,
            authorized=False,
            in_scope=False,
            fresh=True,
            integrity_ok=True,
            policy_admissible=True,
        )
        permitted, _ = operationally_authorized(
            tier=CommunicationTier.T4_OPERATIONAL_ACTIONS,
            envelope=envelope,
        )
        self.assertFalse(permitted)

    def test_non_operational_tier_cannot_execute_even_with_valid_envelope(self):
        envelope = ActionEnvelope(
            authenticated=True,
            authorized=True,
            in_scope=True,
            fresh=True,
            integrity_ok=True,
            policy_admissible=True,
        )
        permitted, _ = operationally_authorized(
            tier=CommunicationTier.T3_CONSTRAINED_EXPERIMENTS,
            envelope=envelope,
        )
        self.assertFalse(permitted)

    def test_delegation_cannot_amplify_capabilities(self):
        result = bounded_delegation(
            parent_capabilities={"read", "write"},
            requested_capabilities={"read", "delete"},
            delegation_scope={"read"},
        )
        self.assertEqual(result.granted, frozenset({"read"}))
        self.assertEqual(result.denied, frozenset({"delete"}))
        self.assertTrue(result.amplified)


if __name__ == "__main__":
    unittest.main()
```